



Desarrollo de Sistemas Móviles

Sesión 5: HTTP y REST

Architecting High-Availability Mobile Apps:
Networking, Concurrency & Offline-First

Al finalizar la clase, serás capaz de:

- **Enterprise Network Layers:** entender la comunicación con Retrofit, OkHttp y Kotlinx Serialization.
- **Advanced Concurrency:** Dominar Kotlin Coroutines y Structured Concurrency para evitar ANRs y memory leaks.
- **Reactive Streams:** Gestionar estados de UI complejos en tiempo real usando Kotlin Flow y StateFlow.
- **Offline-First Paradigm:** Diseñar apps que funcionen sin internet usando una Single Source of Truth (SSOT).
- **Mobile Security:** Blindar la app contra ataques MITM con Certificate Pinning y OAuth 2.1 + PKCE.
- **Real-Time Synchronization:** Conectar Firebase Firestore reactivamente mediante CallbackFlow.
- **Resource Optimization:** Maximizar el rendimiento de memoria y caché de imágenes usando Coil 3.

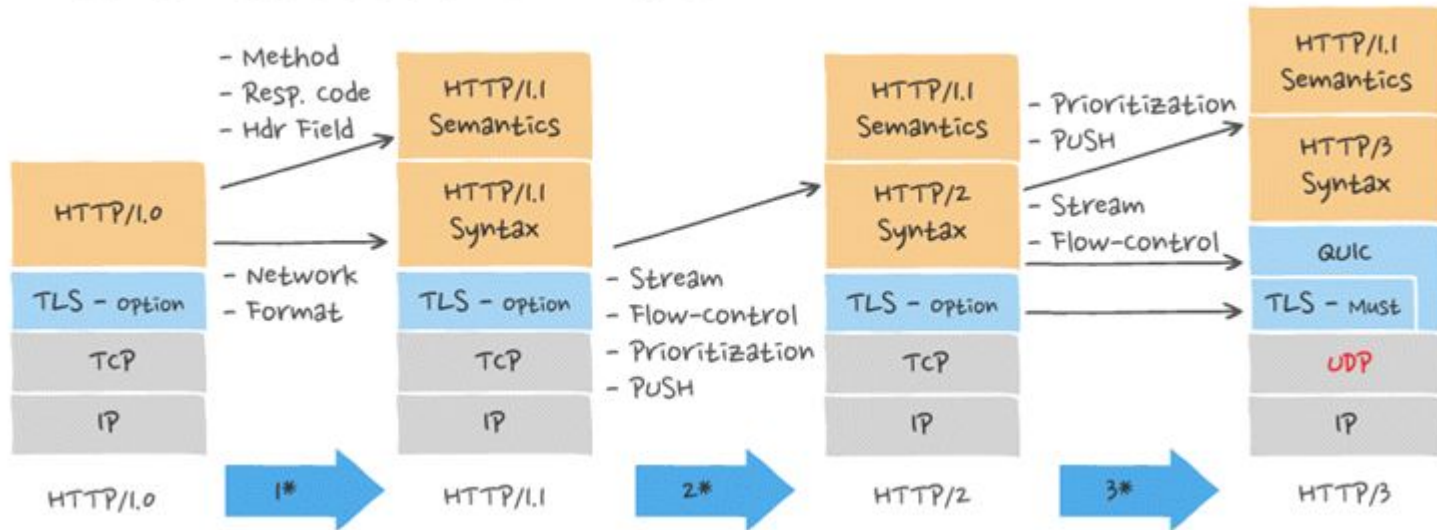
HTTP/3

El protocolo HTTP/3 representa el cambio más drástico en la comunicación de redes móviles de la última década. A diferencia de sus predecesores, abandona el protocolo de transporte TCP en favor de QUIC, el cual está construido sobre UDP.

- Elimination of Head-of-Line Blocking (HoLB):
- **Connection Migration: QUIC** introduce identificadores de conexión independientes de la IP del usuario.
- **Adoption & Integration:** Con más del 35% de adopción entre los sitios web más importantes del mundo, las librerías estándar de Android como OkHttp (4.11+) implementan soporte automático para HTTP/3 siempre que el servidor lo tenga habilitado.

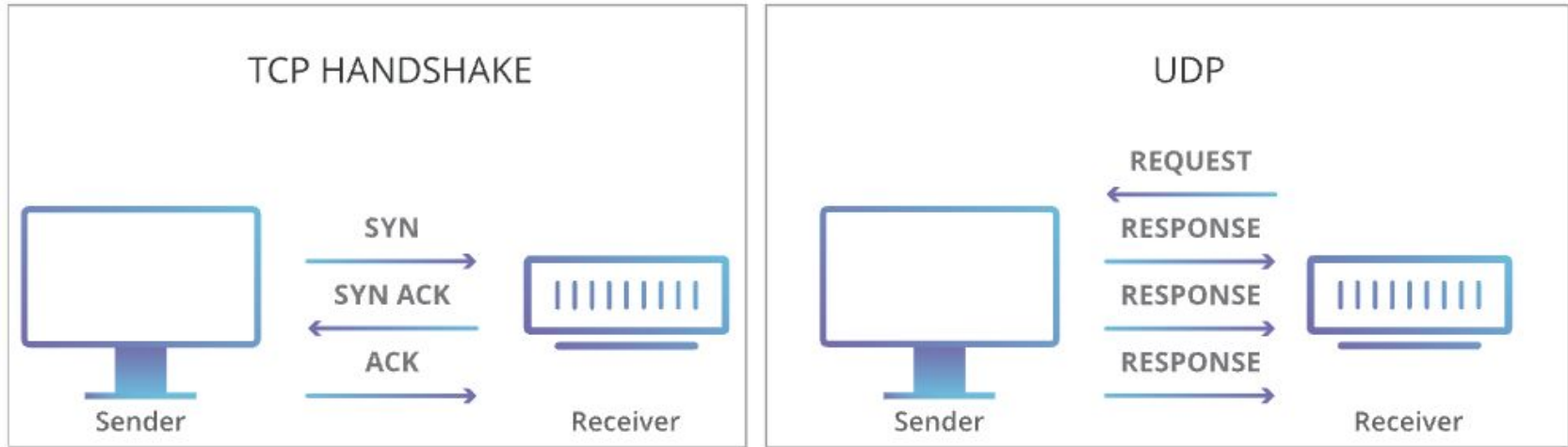
The Evolution to HTTP/3

HTTP protocol stack transition and comparison



User Datagram Protocol: El User Datagram Protocol, o UDP, es un protocolo de comunicación que se utiliza en Internet para transmisiones sujetas a limitaciones temporales, como la reproducción de vídeo o las búsquedas en DNS. Acelera las comunicaciones al no establecer formalmente una conexión antes de transferir los datos. Esto permite que los datos se transfieran muy rápido, pero también puede hacer que los paquetes se pierdan en tránsito y crear oportunidades para vulnerabilidades en forma de ataques DDoS.

TCP vs UDP Communication

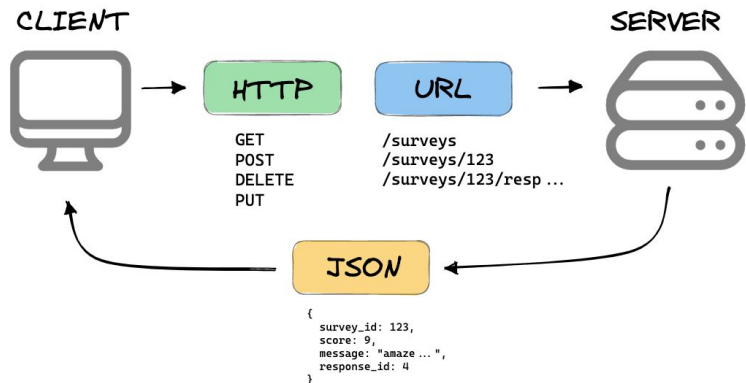


TCP vs UDP Communication

REST Architecture Principles

REST (Representational State Transfer) se mantiene como el estándar empírico para exponer servicios debido a su desacoplamiento y simplicidad

WHAT IS A REST API?



Request:

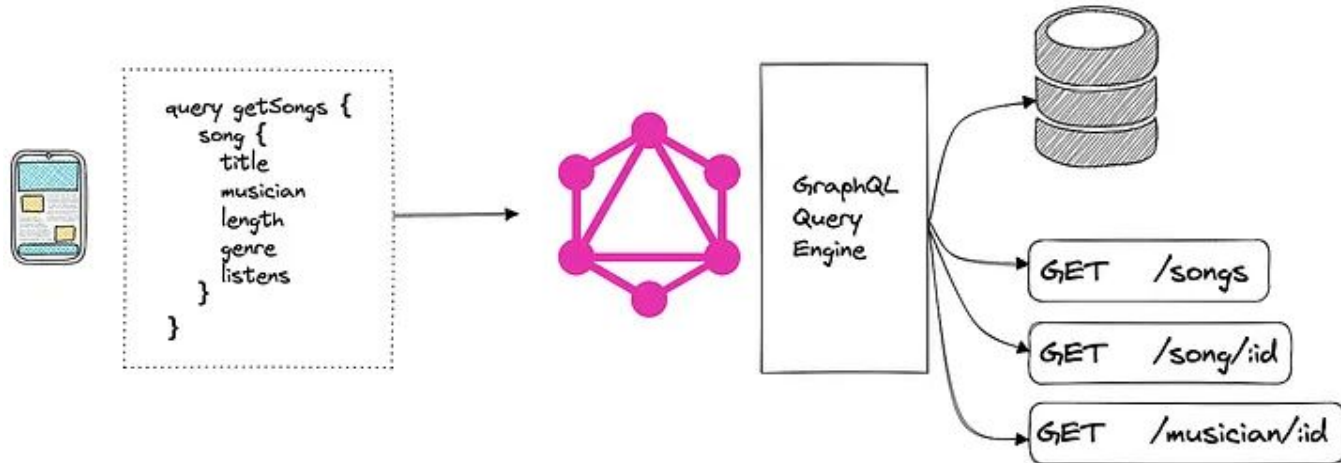
```
GET http://example.com/surveys/123/responses
```

Response:

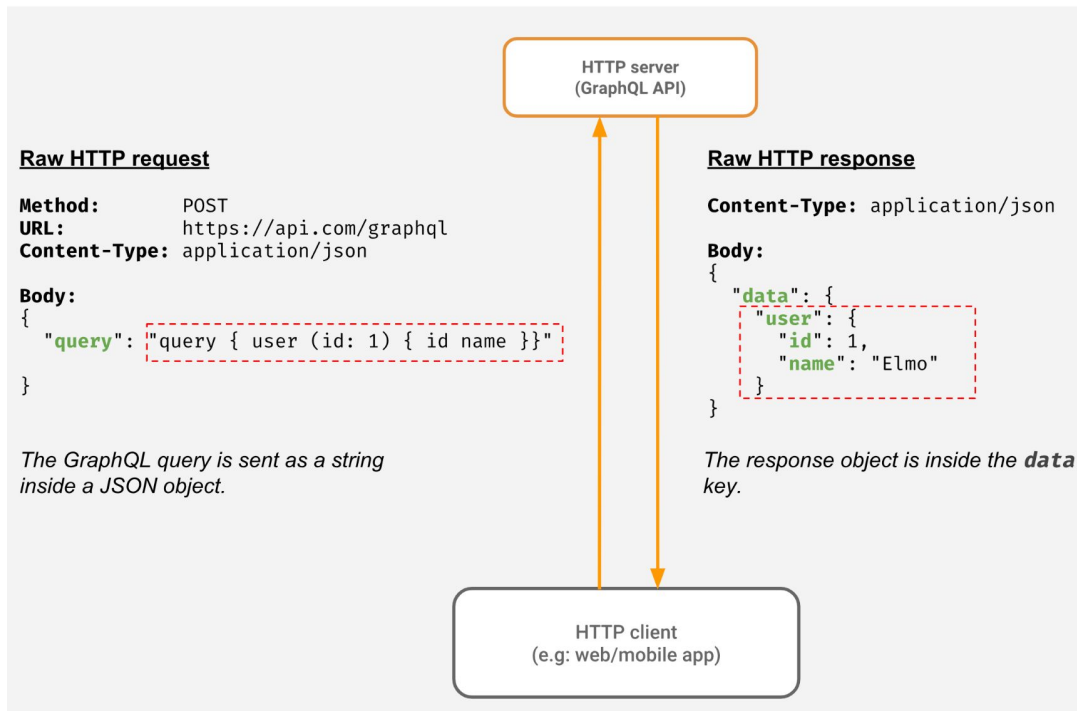
```
// HTTP status code: 200
{
  survey_id: 123,
  survey_title: "nps onboarding survey",
  responses: [
    {
      response_id: 42,
      nps_score: 9,
      feedback: "love the service",
      respondent_id: 42
    }
    ...
  ]
}
```

REST (Multiple Endpoints) vs. GraphQL (Single Endpoint)

GraphQL is an open-source query language for APIs and a server-side runtime. It provides a strongly-typed schema to define relationships between data, making APIs more flexible and predictable. And it isn't tied to a specific database or storage engine — it works with your existing code and data, making it easier to evolve APIs over time.



- **REST Approach (Under-fetching):** El cliente móvil se ve obligado a encadenar múltiples peticiones HTTP:
 - **GET /v1/users/42**
 - **GET /v1/users/42/orders?limit=5**
 - **GET /v1/products/101**
- **GraphQL Approach:** A través de un único endpoint (POST /v1/graphql), la app solicita la forma exacta de los datos que necesita (exact data shape):

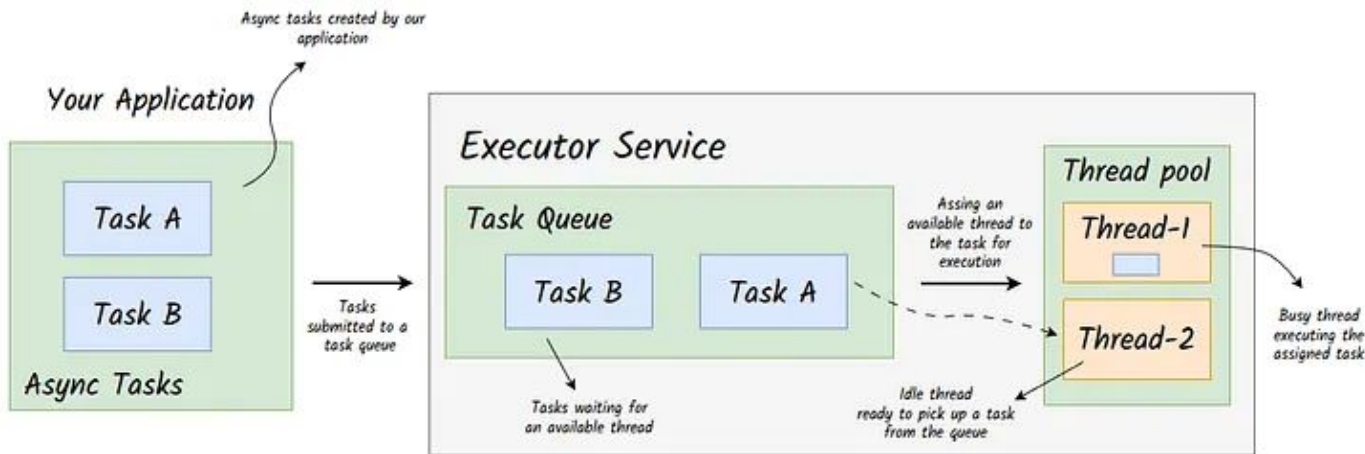




The Main Thread Blocking Problem

En Android, el **Main Thread** se encarga de **renderizar la UI** a 60-120 FPS y de capturar las interacciones del usuario.

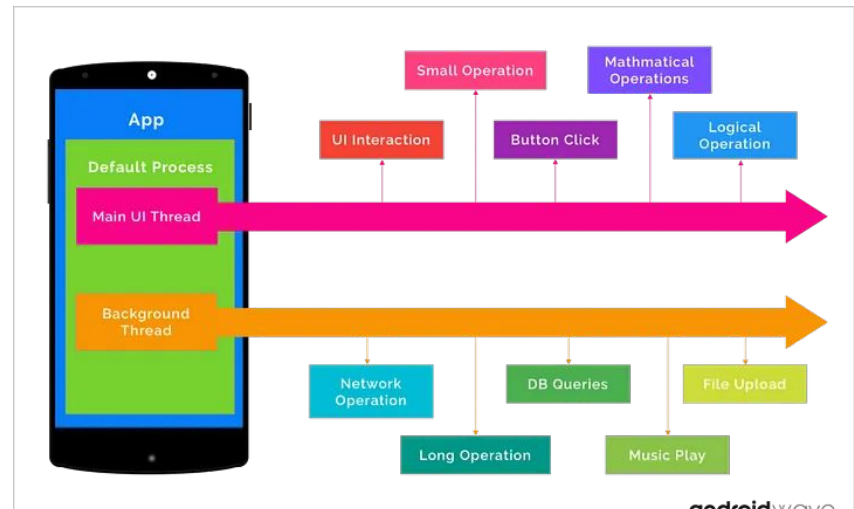
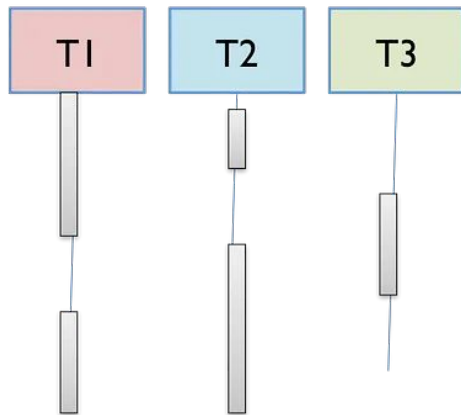
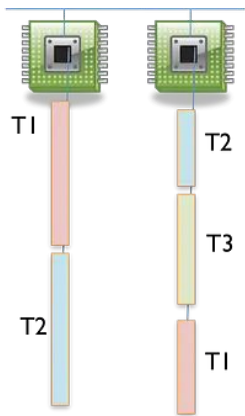
Ejecutar operaciones bloqueantes (blocking operations como network I/O, consultas a bases de datos o heavy serialization) en este hilo dispara excepciones de tipo **ANR** (Application Not Responding).



The Main Thread Blocking Problem

En Android, el **Main Thread** se encarga de **renderizar la UI** a 60-120 FPS y de capturar las interacciones del usuario.

Ejecutar operaciones bloqueantes (blocking operations como network I/O, consultas a bases de datos o heavy serialization) en este hilo dispara excepciones de tipo **ANR** (Application Not Responding).



Structured Concurrency

Las corrutinas no se ejecutan **globalmente** (**GlobalScope** es considerado un anti-pattern). En su lugar, siguen estrictamente un ciclo de vida delimitado por un scope:

- **Scopes:** **viewModelScope** (vinculado al ciclo de vida del ViewModel) garantiza que si un usuario sale de una pantalla, todas las peticiones de red activas dentro de ese scope se cancelan automáticamente, previniendo memory leaks.
- **Dispatchers:** * **Dispatchers.Main:** Operaciones ligeras de UI.
 - **Dispatchers.IO:** Thread pool elástico optimizado para bloquear sockets <https://kotlinlang.org/docs/coroutines-basics.html> de red o almacenamiento.
 - **Dispatchers.Default:** Optimizado para tareas intensivas de CPU (por ejemplo, heavy JSON parsing o procesamiento de imágenes).
- **Parallel Concurrency:** Utilizando `async` y `await()`, puedes disparar múltiples llamadas asíncronas en paralelo, reduciendo el tiempo total de espera al de la petición más lenta.

Structured Concurrency

```
@HiltViewModel
class ProductsViewModel @Inject constructor(private val api: ProductsApi) : ViewModel() {
    ... fun loadScreenContent() {
    ...     viewModelScope.launch { // Coroutine launched on the Main Thread scope
    ...         try {
    ...             // Async calls running concurrently on the IO pool
    ...             val productsDeferred = async(Dispatchers.IO) { api.getProducts() }
    ...             val bannersDeferred = async(Dispatchers.IO) { api.getBanners() }

    ...             // Non-blocking await
    ...             val products = productsDeferred.await()
    ...             val banners = bannersDeferred.await()
    ...             _uiState.value = UiState.Success(products, banners)
    ...         } catch (e: Exception) { _uiState.value = UiState.Error }
    ...     }
    ... }
}
```

Reactive Streams with Kotlin Flow

Flow maneja flujos de datos secuenciales y asíncronos de manera reactiva:

- **Cold Flows**: No consumen recursos ni emiten datos hasta que haya un recolector activo (collect).
- **StateFlow**: Un Hot Flow diseñado para el manejo de estados de UI (UI state management). Siempre retiene el último valor emitido y actúa como un Observer.
- **Transformation Operators**: Puedes orquestar datos utilizando operadores como map, filter, **distinctUntilChanged** (evita re-renderizados si el estado no cambió), y combine para fusionar múltiples flujos en tiempo real.

```
// Real-time search bar filtering an offline-first list
val queryFlow = MutableStateFlow("")
val productsDbFlow: Flow<List<Product>> = dao.getAllFlow()

val filteredProducts: StateFlow<List<Product>> = combine(queryFlow, productsDbFlow) { query, products ->
    ... products.filter { it.isAvailable && it.name.contains(query, ignoreCase = true) }
}.stateIn(scope = viewModelScope, started = SharingStarted.WhileSubscribed(5000), initialValue = emptyList())
```

LAZY OPERATION: son operaciones que realmente se ejecutan cuando haya una acción, esto porque no carga en memoria operaciones previas mientras no se usen.

dataset1: producto, sub-producto, precio **(ya en memoria)**

(transformación) dataset2: producto, precio (max) **# el maximo precio del producto . group by**

(transformación) dataset3: producto, precio (cambiado) **# suma 1 al precio c/p . map . iteracion , ...**

(accion) imprimir: dataset3

imprimir ← **dataset3** ← **dataset2**



Compile-Time Safe Serialization

Las librerías heredadas basadas en reflexión como Gson están obsoletas.

Kotlinx Serialization es el estándar de la industria:

- **Compile-Time Safety:** Valida los esquemas y genera serializadores nativos sin penalizaciones de rendimiento por reflexión en runtime.
- **Strict Null-Safety:** Garantiza el soporte de los tipos anulables (?) de Kotlin y admite valores por defecto (default values) si un campo falta por completo en el payload JSON.

```
@Serializable
data class ProductDto(
    @SerializedName("product_id") val id: Int, // Maps alternative JSON keys
    val name: String,
    val price: Double,
    val description: String? = null, // Safely handles null values
    val category: String = "General" // Default value fallback
)
```

Declarative Network Abstraction with Retrofit

Retrofit abstrae por completo la capa de red transformando interfaces abstractas de Kotlin en implementaciones vivas de clientes **HTTP** a través de anotaciones semánticas (**@GET**, **@POST**, **@Path**, **@Query**, **@Body**).

Al integrar la palabra clave **suspend**, Retrofit se acopla de manera nativa al ecosistema de corrutinas, liberando al programador de gestionar hilos manualmente y retornando colecciones tipadas directamente.

```
interface ProductsApi {  
    ... @GET("v1/products")  
    ... suspend fun getProducts(): List<ProductDto>  
  
    ... @GET("v1/products/{id}")  
    ... suspend fun getProductById(@Path("id") id: Int): ProductDto  
  
    ... @GET("v1/search")  
    ... suspend fun searchProducts(@Query("query") query: String): List<ProductDto>  
  
    ... @POST("v1/products")  
    ... suspend fun createProduct(@Body product: ProductDto): ProductDto  
}
```

OkHttp Interceptors

OkHttp actúa como el motor subyacente de Retrofit.

A través de su arquitectura de Interceptors, inyecta lógica transversal de forma global:

- **Logging Interceptor:** Imprime condicionalmente las cabeceras (**headers**) y los cuerpos de carga (**payloads**) solo durante el desarrollo (**BuildConfig.DEBUG**).
- **Authentication Interceptor:** Intercepta las peticiones salientes para adjuntar cabeceras globales como el Bearer Token.

```
val authInterceptor = Interceptor { chain ->
    ... val request = chain.request().newBuilder()
    ...     .addHeader("Authorization", "Bearer ${tokenStore.getToken()}")
    ...     .build()
    ... chain.proceed(request)
}
```

POST Login school GET Me (school us POST Selecciona GET Listar roles dis GET Listar usuario GET Listar permiso POST Login Hamutay API GET Me

Hamutay API > Platform > Auth > Login

POST {{base_url}} /platform/auth/login

Docs Params Authorization Headers 9 Body Scripts Settings

Pre-request 1 var res = pm.response.json();
Post-response 2 if (res.access_token) { pm.collectionVariables.set('platform_token', res.access_token); }

POST Login school GET Me (school us POST Selecciona GET Listar roles dis GET Listar usuario GET Listar permiso POST Login Hamutay API GET Me Platform +

Hamutay API > Platform > Auth > Me Save Share

GET {{base_url}} /platform/auth/me Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Headers Hide auto-generated headers

Key	Value	Description	Bulk Edit	Presets	...
✓ Postman-Token	<calculated when request is sent>				
✓ Host	<calculated when request is sent>				
✓ User-Agent	PostmanRuntime/7.53.0				
✓ Accept	*/*				
✓ Accept-Encoding	gzip, deflate, br				
✓ Connection	keep-alive				
✓ Authorization	Bearer {{platform_token}}				
Key	Value	Description			

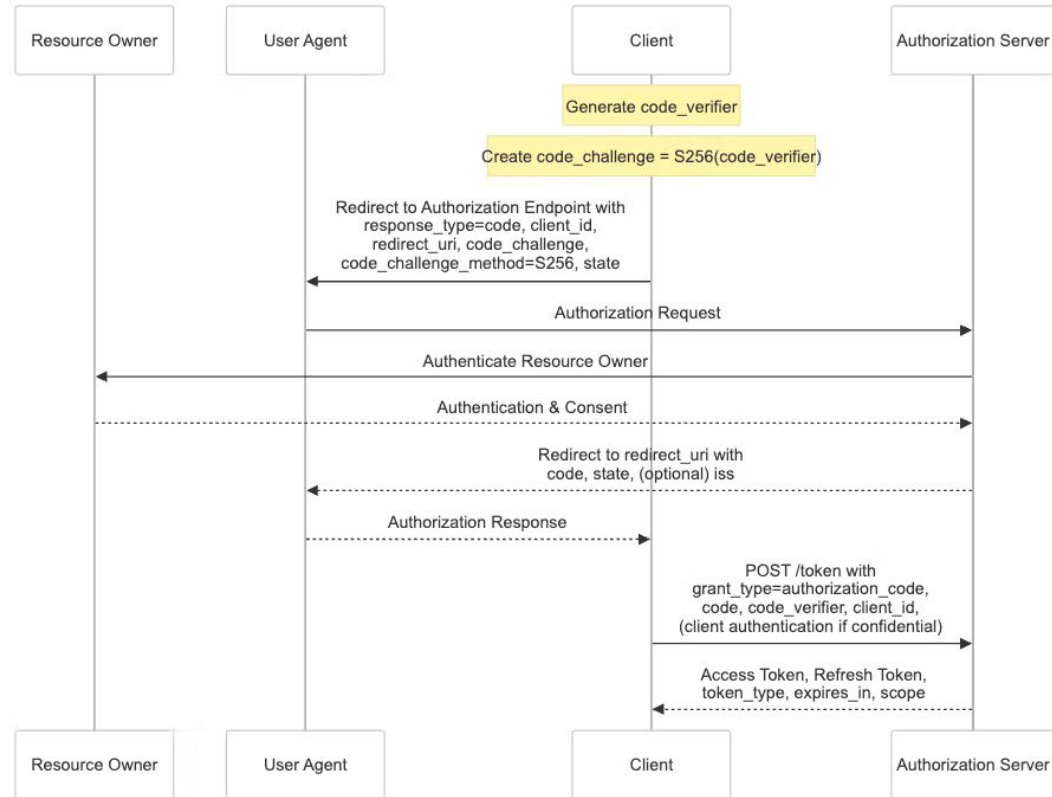


OAuth 2.1 & Mandated PKCE

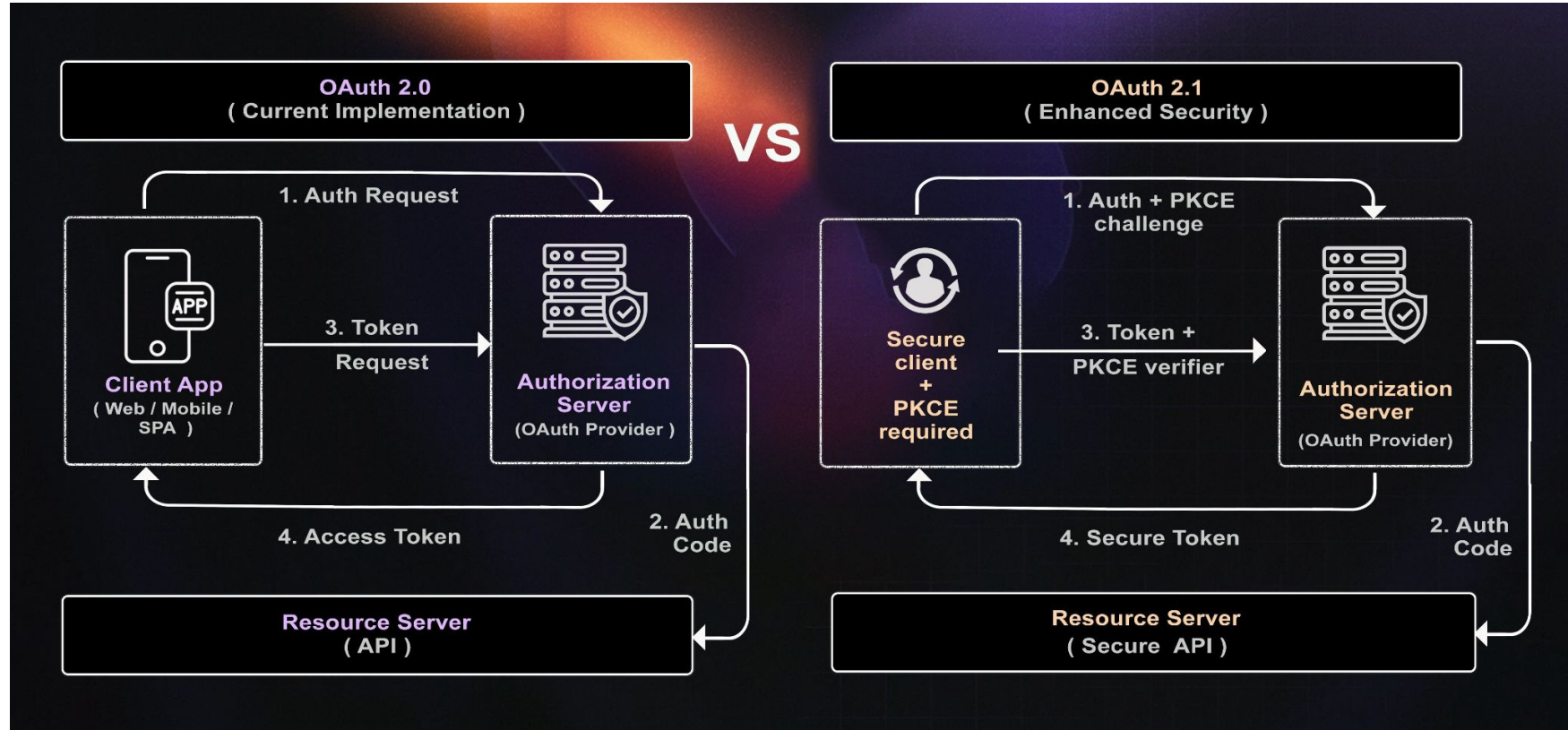
Para aplicaciones móviles (public clients que no pueden ocultar de forma segura un client secret), **OAuth 2.1** hace mandatorio el uso de **PKCE (Proof Key for Code Exchange)**.

La app genera un **code_verifier** aleatorio y envía su versión hasheada (**code_challenge**) durante la autorización.

Esto evita que aplicaciones maliciosas intercepten el authorization code a través de custom **URI** schemes en el sistema operativo.



Enterprise-Grade Security in Communications UNMSM

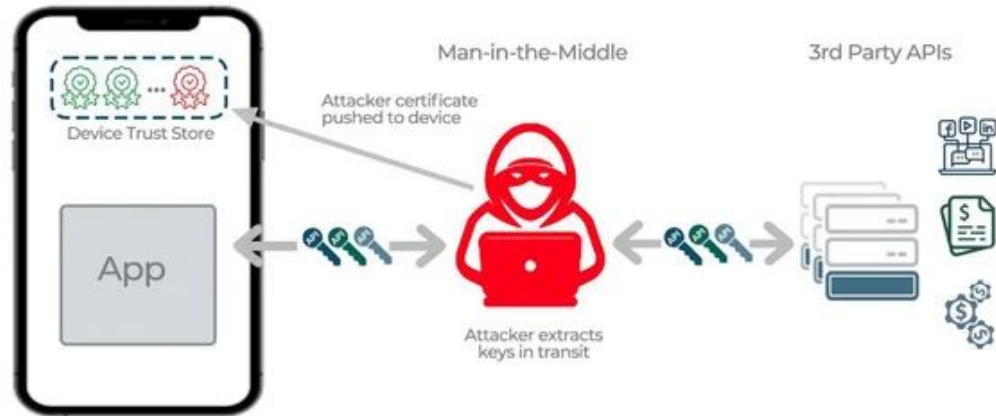


Certificate Pinning

Para eliminar los ataques **Man-in-the-Middle (MITM)**, donde un atacante inyecta un certificado raíz comprometido en el dispositivo del usuario, implementamos **Certificate Pinning**.

Al codificar rígidamente (**hardcoding**) los hashes **SHA-256** de la clave pública del servidor directamente en el cliente **OkHttp**, la app rechazará instantáneamente cualquier **TLS handshake** si la cadena de certificados no coincide.

```
val client = OkHttpClient.Builder()
    .certificatePinner(CertificatePinner.Builder()
        .add(
            "api.miapp.com",
            "sha256/7nc8xxxxxxxx+Yh4MxVxxxxxx/M8="
        )
        .build()
    ).build()
```



Refresh Token Rotation

Para mitigar los riesgos de robo de tokens, se implementa una estrategia de vida corta y vida larga (short-lived/long-lived):

- El Access Token tiene una vida útil corta (por ejemplo, 15 minutos).
- Cuando una petición devuelve un estado 401 Unauthorized, un interceptor de tipo Authenticator de OkHttp pausa la cola de peticiones.
- Realiza una llamada sincrónica para intercambiar el Refresh Token.
- El servidor invalida el antiguo Refresh Token, emite un nuevo par (Refresh Token Rotation), y el interceptor reintenta la petición original de manera transparente para el usuario.



El paradigma Offline-First establece que una aplicación debe permanecer completamente operativa sin una conexión a internet, confiando en la persistencia local.

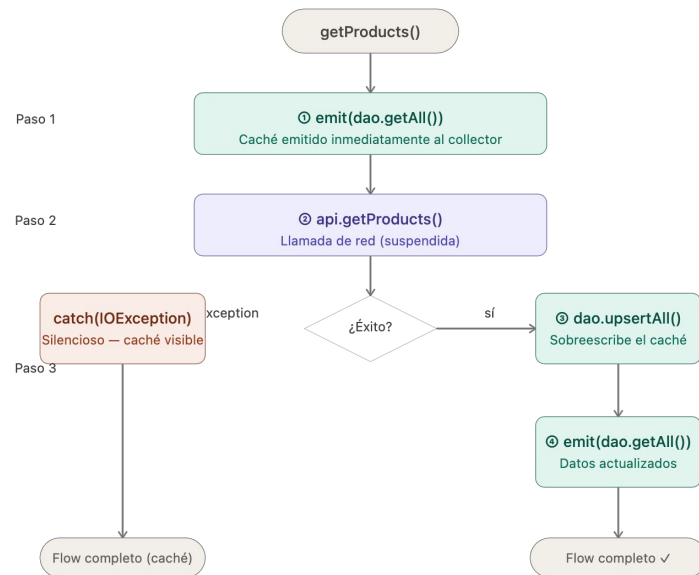
Cache-Then-Network Strategy

El repositorio sirve como la única fuente de verdad (Single Source of Truth), orquestando los flujos de datos a través de Kotlin Flow:

- **Immediate Local Emission:** El repositorio consulta la caché de la base de datos local (Room) y la emite instantáneamente a la UI, evitando pantallas de carga vacías.
- **Asynchronous Network Fetch:** Simultaneously, dispara una petición HTTP vía Retrofit para traer datos frescos del backend.
- **Persistence Overwrite:** Tras una respuesta de red exitosa, los nuevos datos se guardan en la base de datos utilizando una estrategia **Upsert**. Room detecta automáticamente el cambio en la tabla y propaga las actualizaciones al flujo de la UI. Si ocurre un error de red (IOException), se captura silenciosamente (silent catch), dejando los datos cacheados intactos en la pantalla.

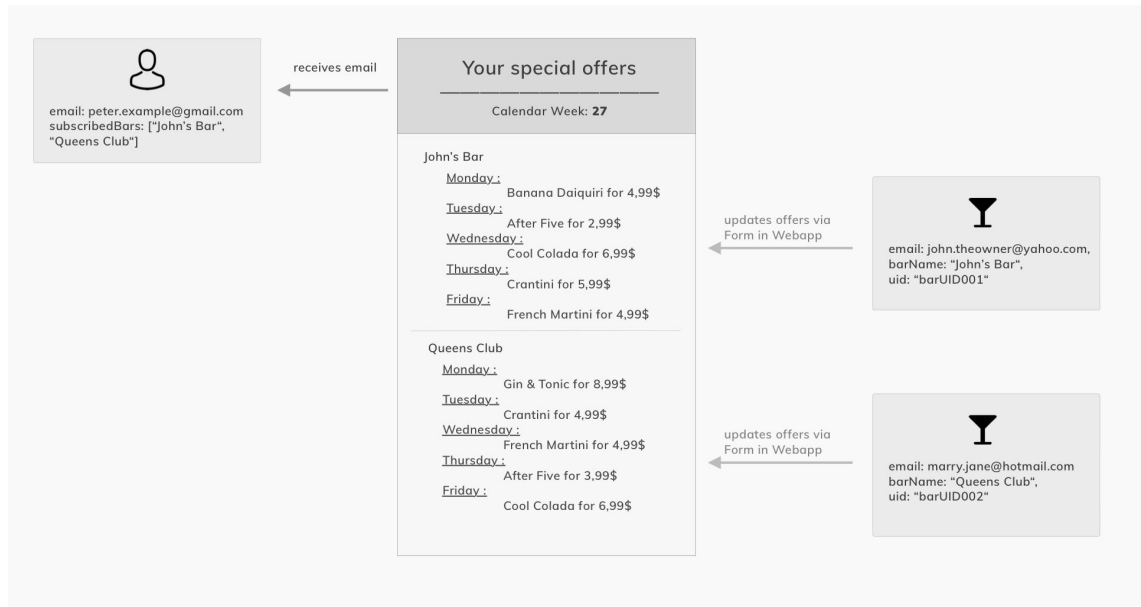
Cache-Then-Network Strategy

```
override fun getProducts(): Flow<List<Product>> = flow {  
    emit(dao.getAll().map { it.toDomain() }) // Step 1: Immediate Cache Emit  
    try {  
        val remote = api.getProducts() // Step 2: Network Fetch  
        dao.upsertAll(remote.map { it.toEntity() }) // Step 3: Cache Overwrite  
        emit(dao.getAll().map { it.toDomain() })  
    } catch (e: IOException) { /* Silent catch, cache remains visible */ }  
}
```



NoSQL Document Structure

Firebase Firestore organiza los datos jerárquicamente en Collections y Documents. Los documentos son estructuras JSON binarias que favorecen la desnormalización de datos (data denormalization) para garantizar lecturas ultrarrápidas a gran escala.



Reactive Integration with callbackFlow

Mientras que las operaciones atómicas utilizan `.await()` como funciones de suspensión estándar, la sincronización en tiempo real se acopla a los Snapshot Listeners de Firestore dentro de un **callbackFlow** para transformar los eventos de la nube en un stream reactivo de Kotlin:

mail@abc.com → abc.mail@abc.com

```
fun streamProducts(): Flow<List<Product>> = callbackFlow {  
    val listener = firestore.collection("products")  
        .addSnapshotListener { snapshot, error ->  
            if (error != null) { close(error); return@addSnapshotListener }  
  
            val products = snapshot?.documents?.mapNotNull {  
                it.toObject<ProductDto>()?.toDomain()  
            } ?: emptyList()  
  
            trySend(products) // Pushes data into the Flow stream  
        }  
    awaitClose { listener.remove() } // Critical memory-leak cleanup  
}
```



Coil 3 (Coroutines Image Loader) es el estándar moderno para la carga de imágenes en Jetpack Compose. Resuelve la asignación de memoria de los bitmaps de forma transparente:

Cache-Then-Network Strategy

El repositorio sirve como la única fuente de verdad (Single Source of Truth), orquestando los flujos de datos a través de Kotlin Flow:

- **Asynchronous Pipeline:** Utiliza Kotlin Coroutines para manejar la descarga, decodificación y redimensionamiento de imágenes fuera del hilo principal (Dispatchers.IO), previniendo el UI jank (micro-tirones).
- **Double-Layer Caching:** Orquesta automáticamente una caché en memoria RAM (para acceso inmediato durante el scrolling de listas) y una caché en disco a través de OkHttp, minimizando el consumo de datos celulares del usuario. [Internet] → [Caché en Disco (OkHttp)] → [Caché en RAM] → [Tu Pantalla]

```
AsyncImage(  
    model = ImageRequest.Builder(LocalContext.current)  
        .data("https://example.com/img.jpg")  
        .crossfade(true) // Smooth visual transition  
        .build(),  
    contentDescription = "Product Image",  
    placeholder = painterResource(R.drawable.skeleton_placeholder), // Shimmer/Skeleton  
    error = painterResource(R.drawable.network_error_holder) // Network fallback  
)
```

Lab

Gracias

